

17: Parallelism II (and Pipelining I)

ENGR 315: Hardware/Software CoDesign

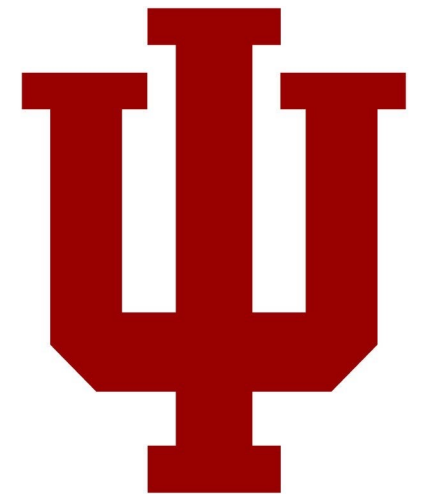
Andrew Lukefahr

Indiana University

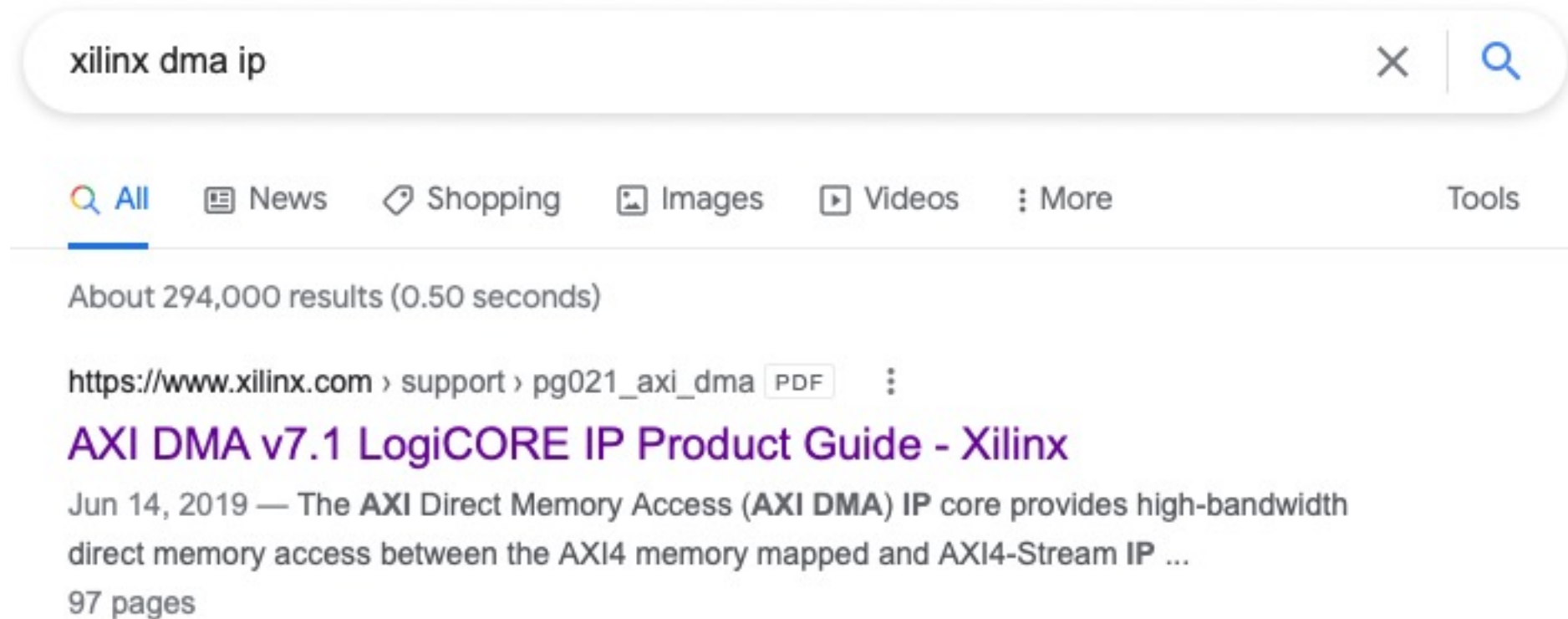
Some material taken from:

https://github.com/trekhleb/homemade-machine-learning/tree/master/homemade/neural_network

<http://cs231n.github.io/neural-networks-1/>



P7 – DMA from C



P7 – DMA from C

Programming Sequence

Direct Register Mode (Simple DMA)

P7 – DMA from C

1. Start the MM2S channel running by setting the run/stop bit to 1 (MM2S_DMACR.RS = 1). The halted bit (DMASR.Halted) should deassert indicating the MM2S channel is running.

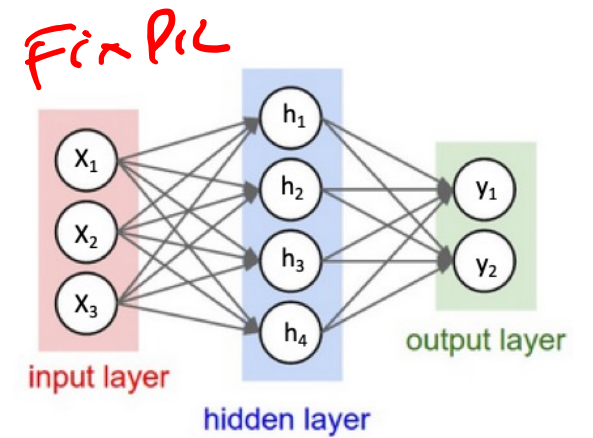
2. Skip

3. Write a valid source address to the MM2S_SA register.
4. Write the number of bytes to transfer in the MM2S_LENGTH register.
The MM2S_LENGTH register must be written last.
5. Wait until MM2S_DMASR.Idle==1 for completion

P8+ are part of the “Design Challenge”

- Goal: Accelerate reference neural network
- Harder, more open-ended projects

P8+ focus on Dot Product



```
# forward-pass of a 3-layer neural network:  
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)  
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)  
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)  
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)  
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

Matrix Multiplication (Dot Product)

$$\begin{array}{c} \text{inputs} \\ \underline{[0.1 \quad 0.2]} \end{array} \times \begin{array}{c} \text{weights} \\ \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \end{array} =$$

$$= \begin{bmatrix} (0.1 \times 1 + 0.2 \times 4) & (0.1 \times 2 + 0.2 \times 5) & (0.1 \times 3 + 0.2 \times 6) \end{bmatrix}$$

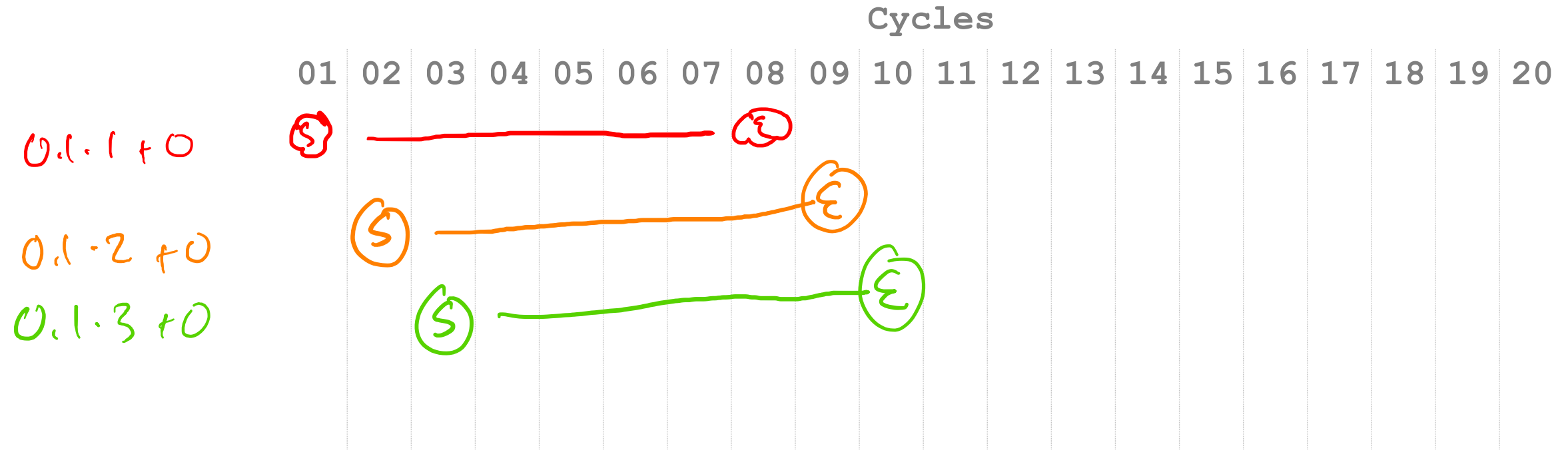
$$= \begin{array}{c} \text{(Answer)} \\ \underline{[0.9 \quad 1.2 \quad 1.5]} \end{array}$$

Floating-Point math takes 8 cycles.

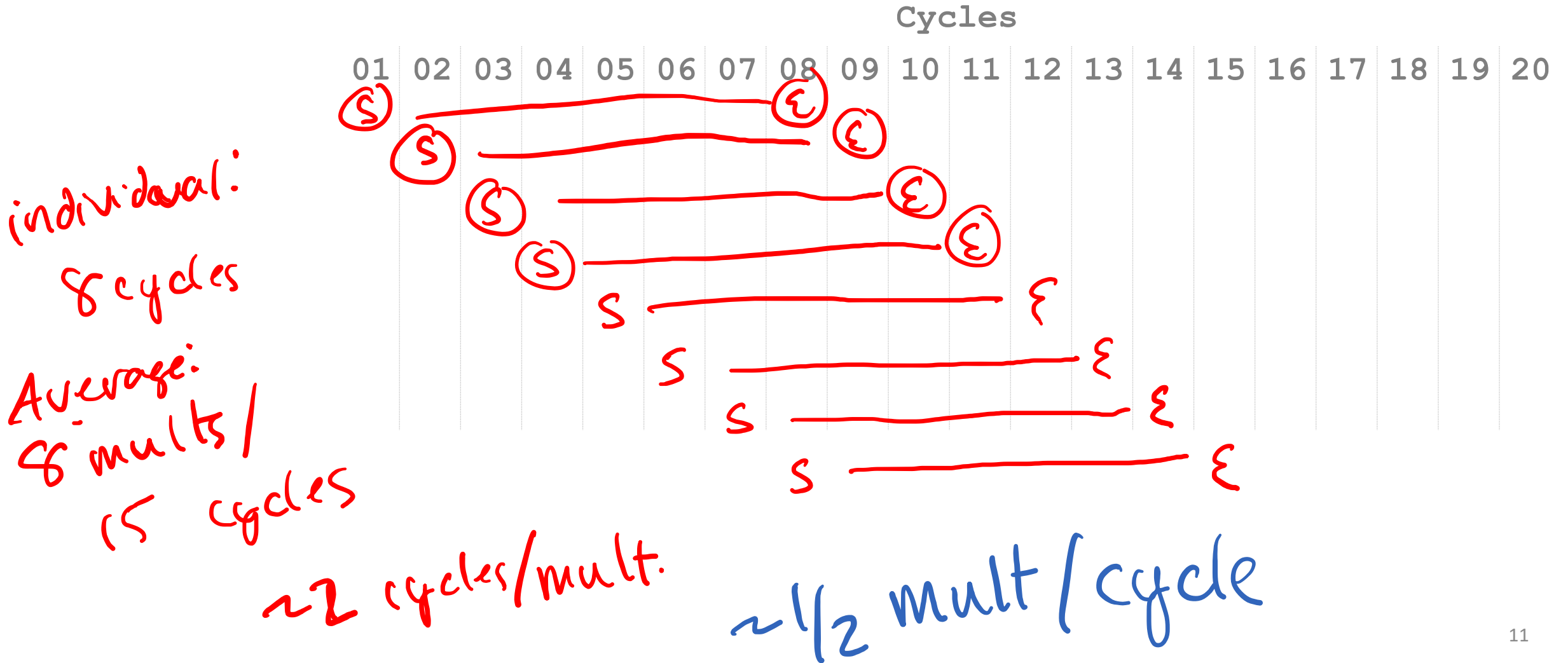
- Floating-Point is complicated.
- FMAC: $a * b + c$
- 8 cycles of complicated.
- How do we work around an 8 cycle latency?
- Pipelining!

FMAC Pipelining

$$\begin{bmatrix} 0.1 & 0.2 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$



FMAC Pipelining



Latency vs. Throughput

- **Latency:** How long does an individual operation take to complete?

individual mult: 8 cycles

- **Throughput:** How many operations can you complete per second (or per cycle)?

Average output per cycles:

~ 1 mult / cycle

Multiply-Accumulate Dot Computations

$$\begin{bmatrix} 0.1 & 0.2 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} = \begin{bmatrix} 0.9 & 1.2 & 1.5 \end{bmatrix}$$

$= \begin{bmatrix} \underline{0} & \underline{0} & \underline{0} \end{bmatrix}$

$$0.1 \cdot 1 + 0 \quad 0.1 \cdot 2 + 0 \quad 0.1 \cdot 3 + 0 = \begin{bmatrix} 0.1 & \underline{0.2} & \underline{0.3} \end{bmatrix}$$

$$0.2 \cdot 4 + 0.1 \quad 0.2 \cdot 5 + 0.2 \quad 0.2 \cdot 6 + 0.3 = \begin{bmatrix} 0.9 & 1.2 & 1.5 \end{bmatrix}$$

Dependencies

$$\begin{bmatrix} 0.1 & 0.2 \end{bmatrix} \times \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

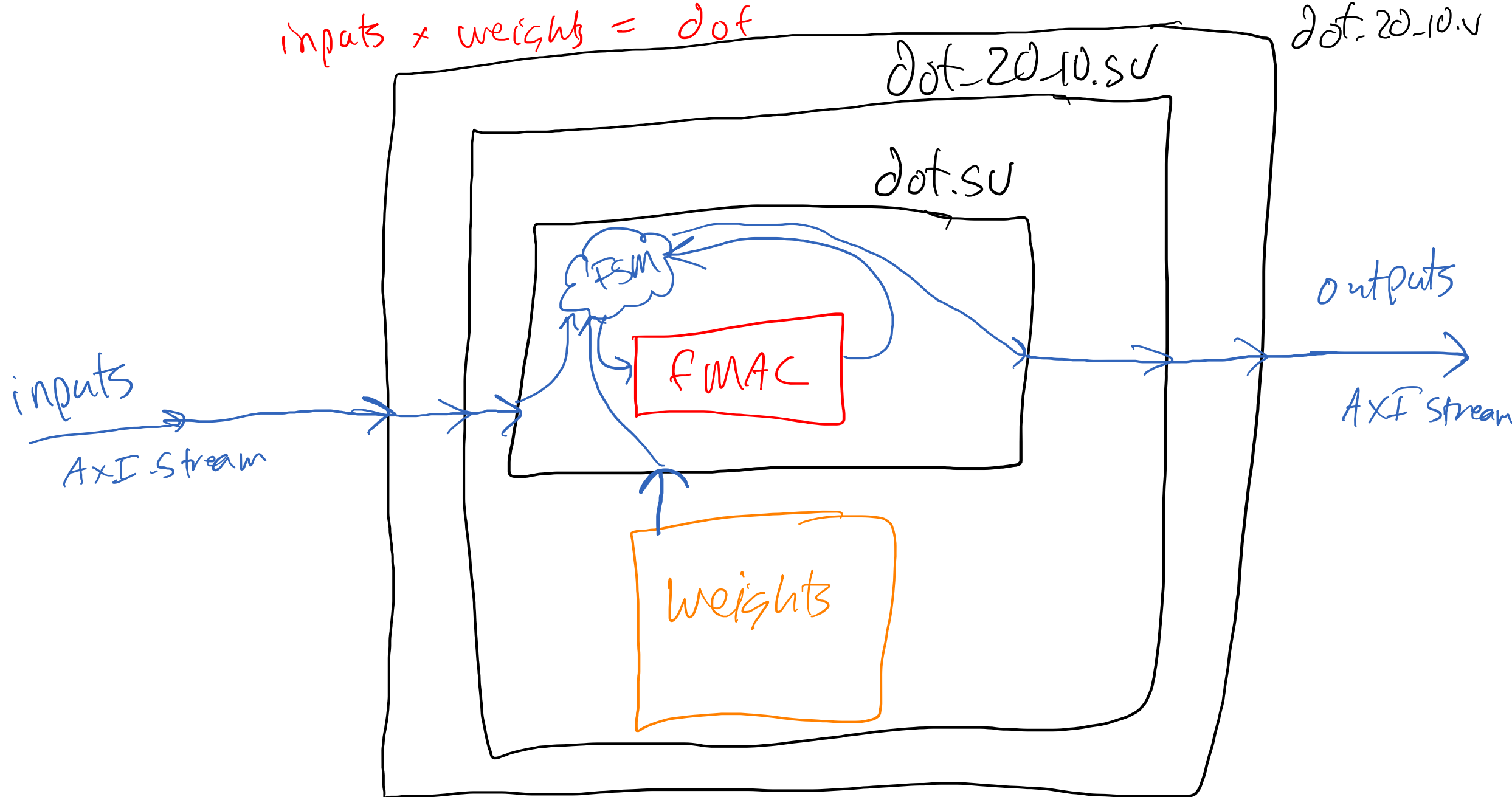
Cycles

Category	Cycles
01	10
02	20
03	30
04	40
05	50
06	60
07	70
08	80
09	90
10	100
11	90
12	80
13	70
14	60
15	50
16	40
17	30
18	20
19	10
20	0

Dependencies on Pipelined FMAC

- Solution: Stall at the end of a row.
- “Drain” the pipeline.
- Restart new row
- “Refill” the pipeline

inputs * weights = dot



5.05

How many cycles should this take?

$$\begin{bmatrix} 0.1 & 0.2 & 0.3 \end{bmatrix} \begin{bmatrix} \begin{matrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{matrix} \end{bmatrix} = \begin{bmatrix} 3.8 & 4.4 & 5 & 5.6 \end{bmatrix}$$

8 cycles 1 multiply x 12 multipliers
= 96 cycles

Data flow graph

- A data-flow graph is a **collection of arcs and nodes in which the nodes are either places where variables are assigned or used**, and the arcs show the relationship between the places where a variable is assigned and where the assigned value is subsequently used.

[\[ref\]](#)

Data Flow Graph

- Illustrates the dependencies between inputs and operations to produce an output



Inputs



Operations

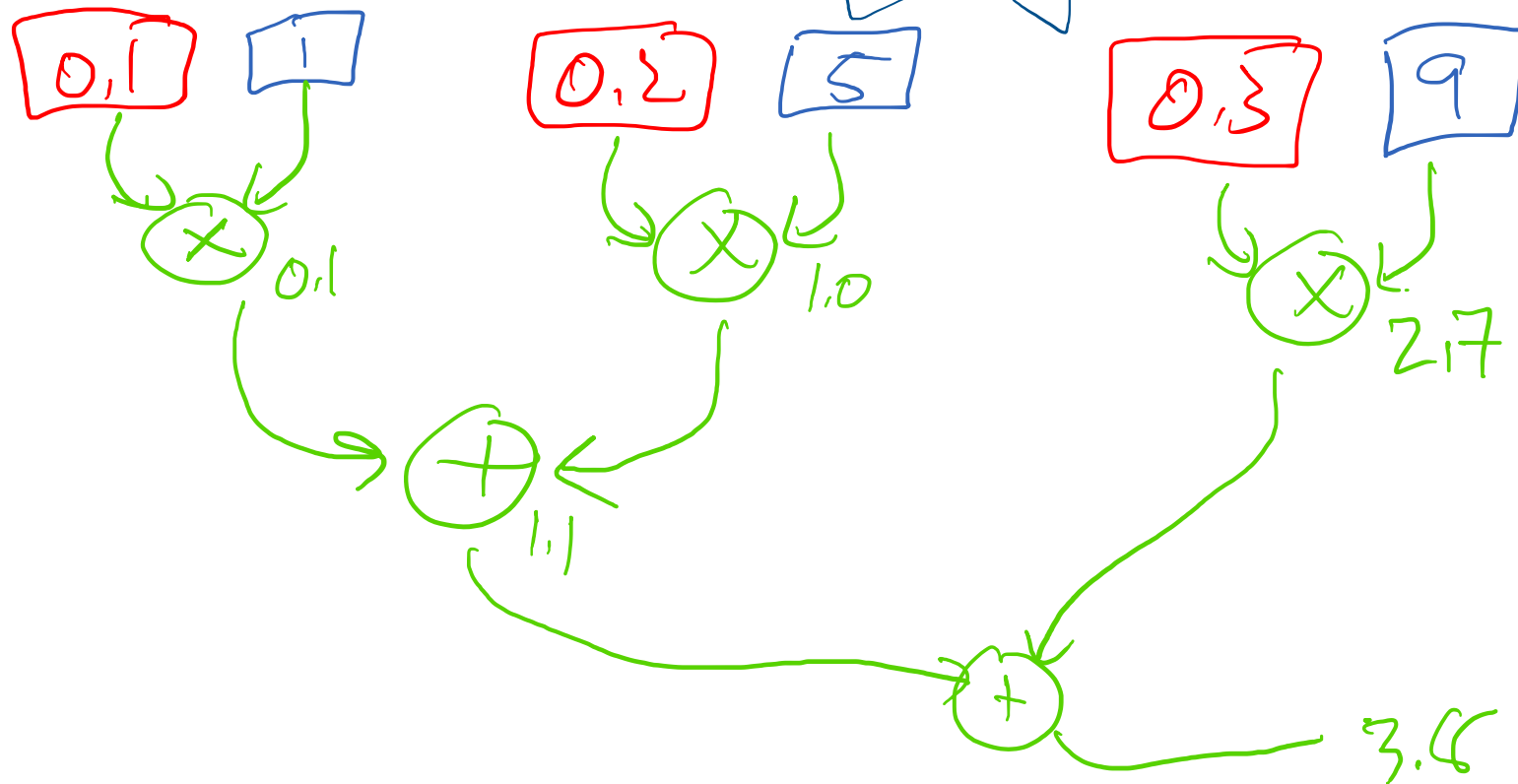


Dependencies

Data Flow Graph

≠ NO MAC for NOW
≠ NO pipeline

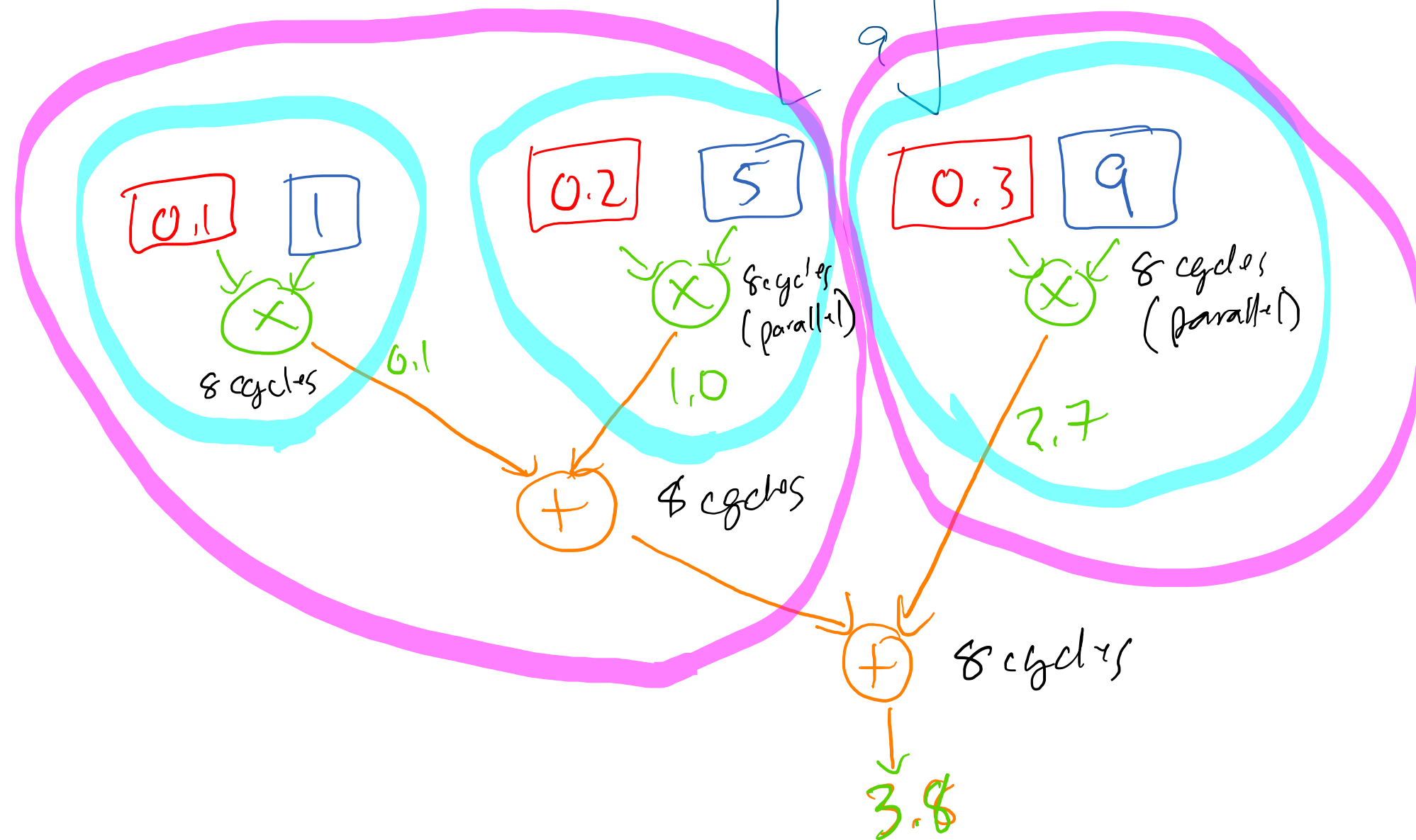
$$\begin{bmatrix} 0.1 & 0.2 & 0.3 \end{bmatrix} \begin{bmatrix} 1 \\ 5 \\ 9 \end{bmatrix} = \begin{bmatrix} 3.8 \end{bmatrix}$$



Finding Parallelism

- Find some computation that doesn't depend on other computation's results.
- No arrows between blocks.
- Shared Inputs are OK.

$$\begin{bmatrix} 0.1 & 0.2 & 0.3 \end{bmatrix} \begin{bmatrix} 1 \\ 5 \\ 9 \end{bmatrix} = \begin{bmatrix} 3.8 \end{bmatrix}$$



Infinite Hardware

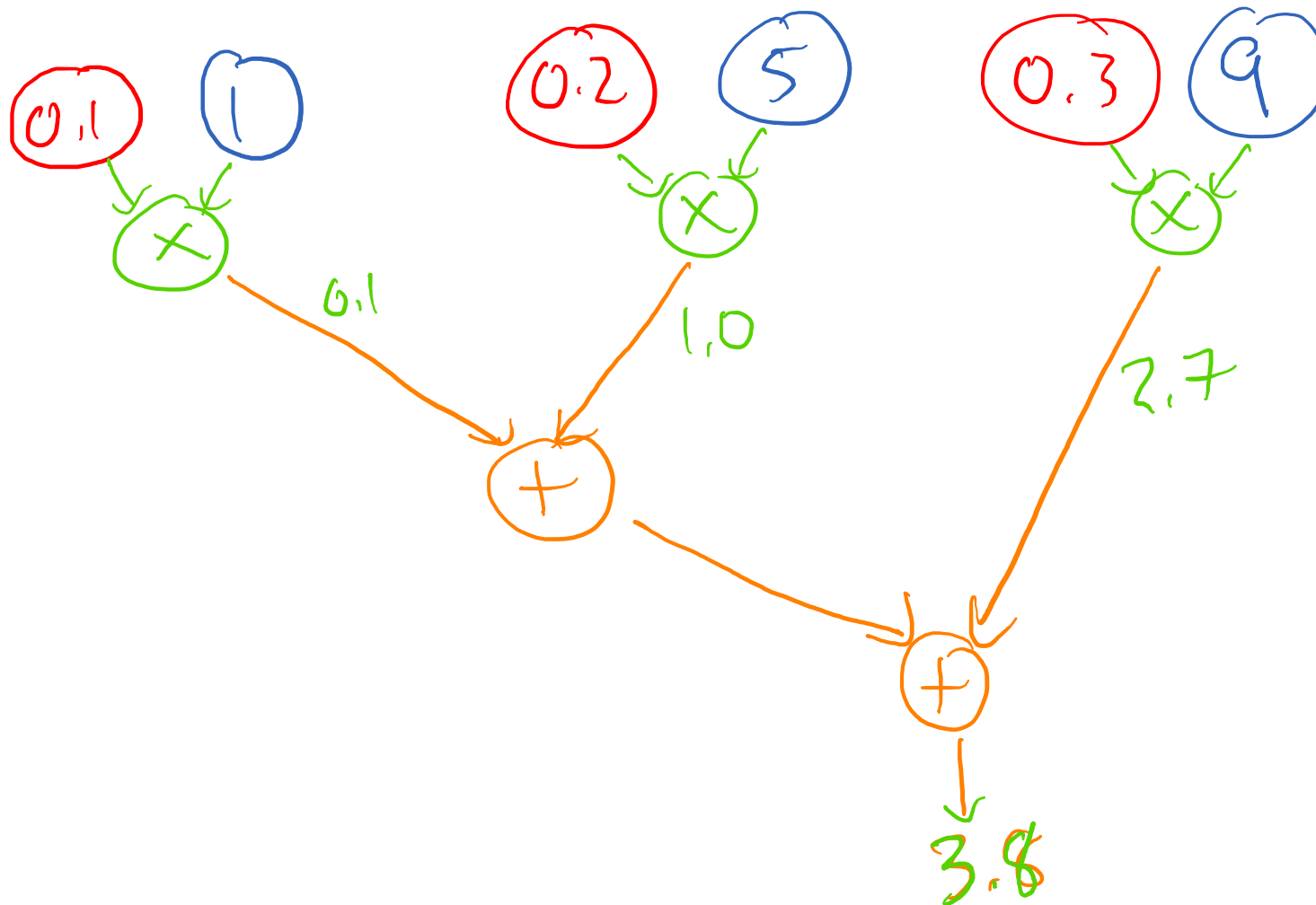
← 8 cycles

← 8 cycles

← 8 cycles

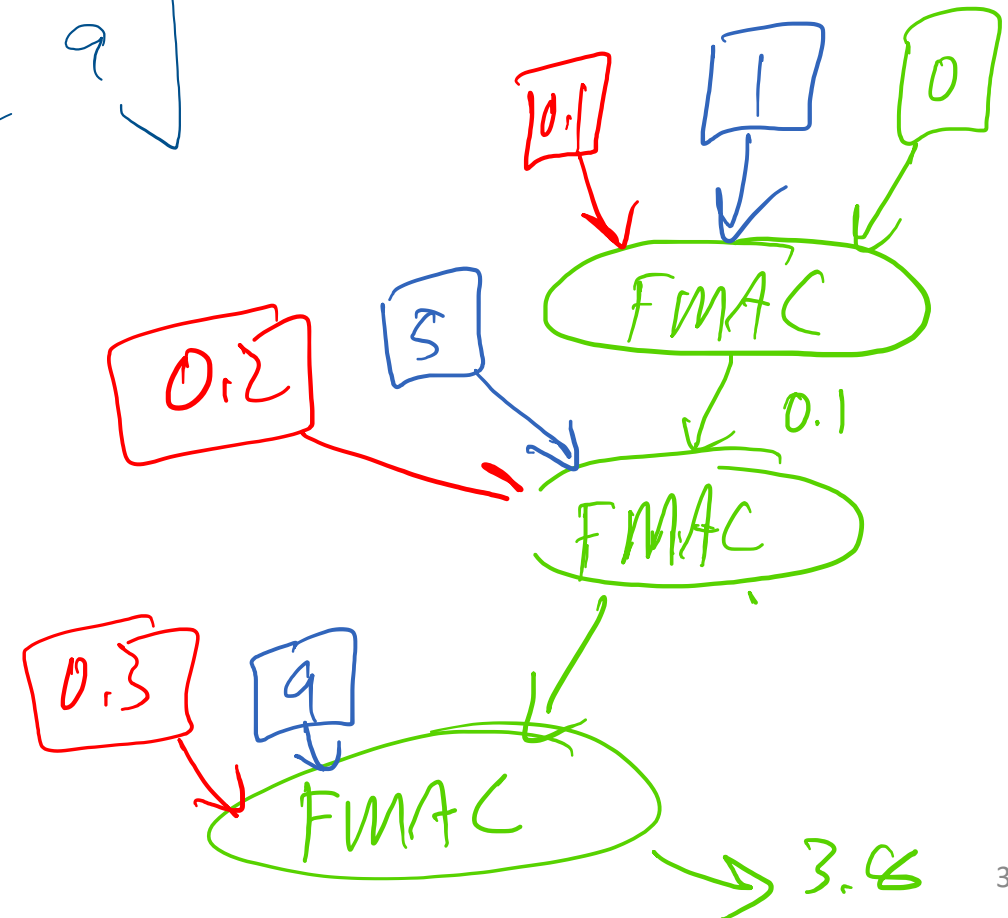
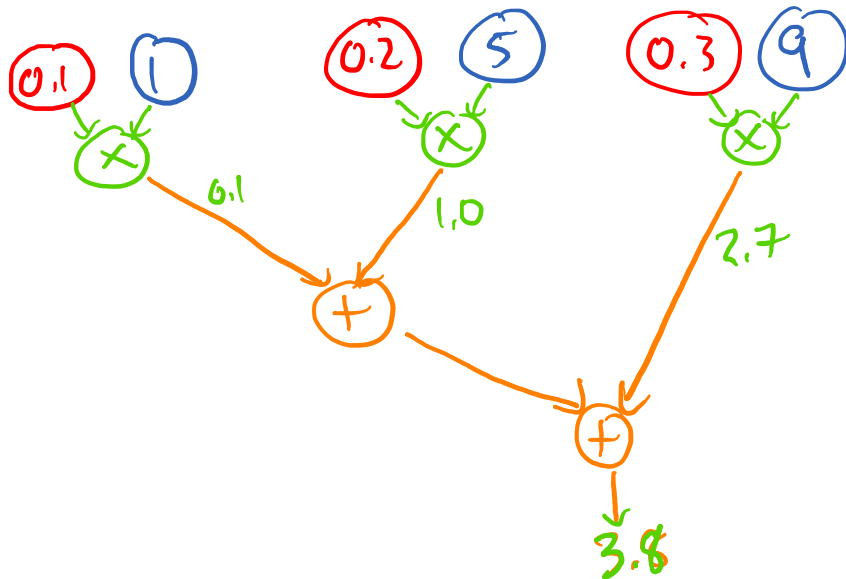
 24 cycles

How many cycles should this take?



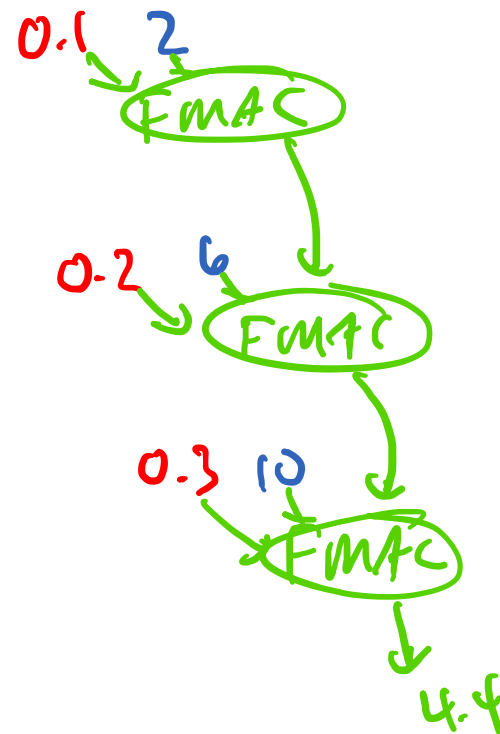
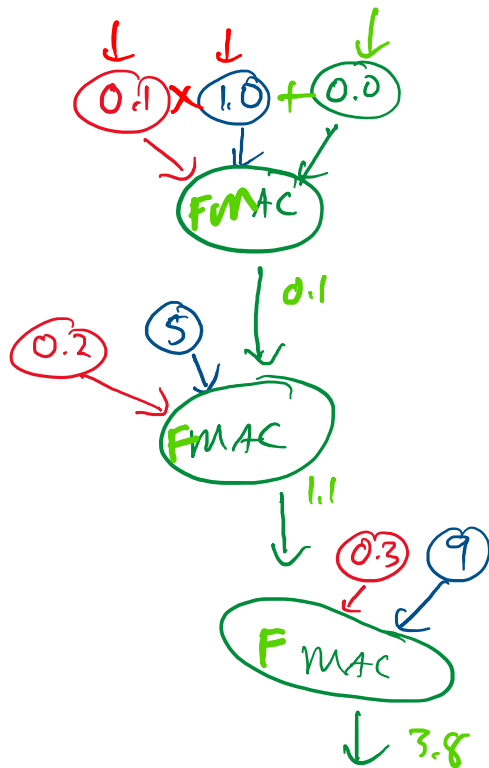
How many cycles should this take?

$$\begin{bmatrix} 0.1 & 0.2 & 0.3 \end{bmatrix} \begin{bmatrix} 1 \\ 5 \\ 9 \end{bmatrix} = \begin{bmatrix} 3.8 \end{bmatrix}$$



More Data Flow

$$\begin{bmatrix} 0.1 & 0.2 & 0.3 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix} = \begin{bmatrix} 3.8 & 4.4 & 5 & 5.6 \end{bmatrix}$$

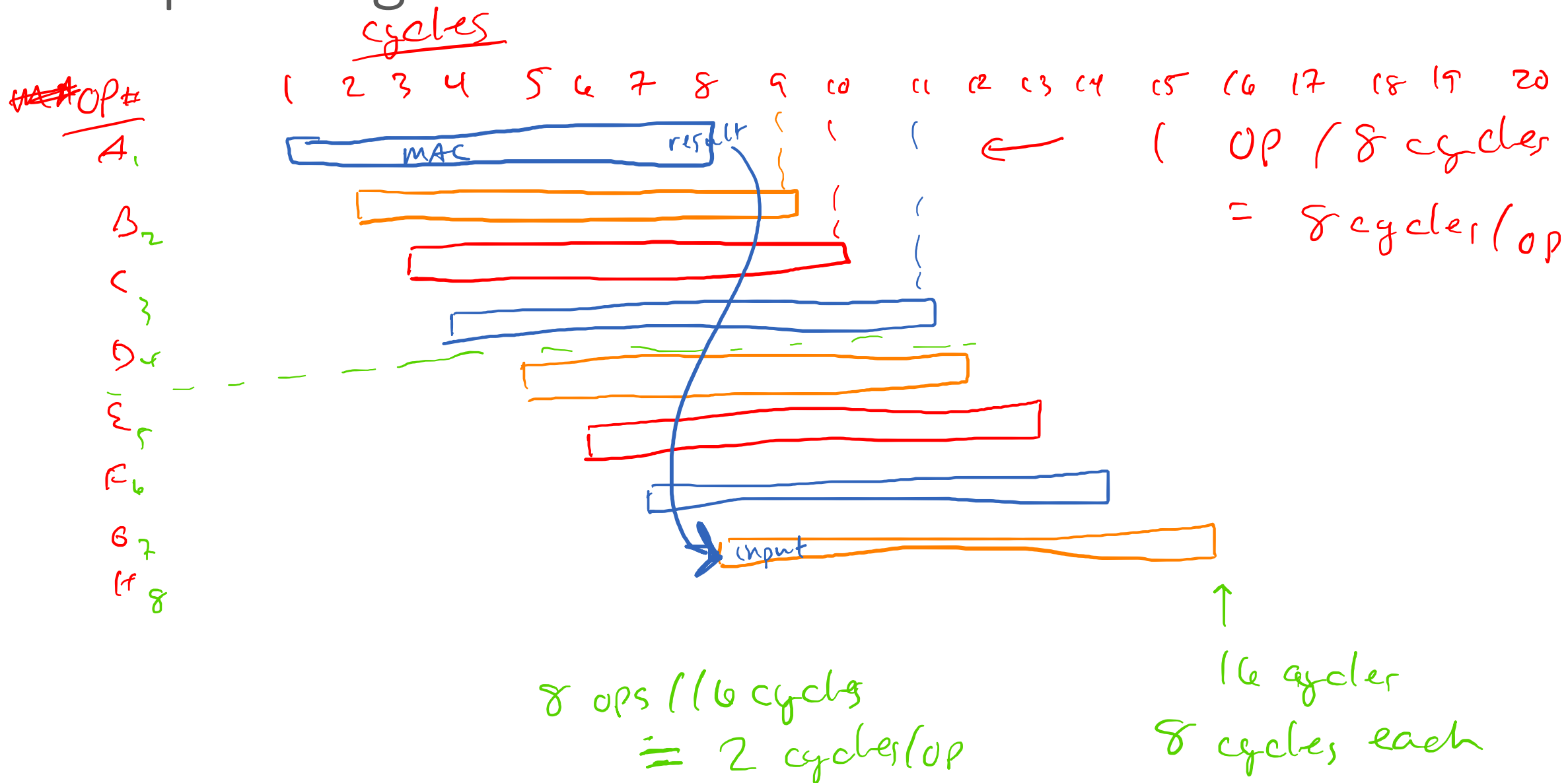


Pipelining

- FMAC takes 8 cycles for 1 value
- But can accept a new value every cycle.

- Latency: 8 cycles / value ↩
- Throughput: 1 value / cycle

Pipelining



$$\begin{bmatrix} \underline{0.1} & 0.2 & 0.3 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 \\ \underline{5} & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix} = \begin{bmatrix} 3.8 & 4.4 & 5 & 5.6 \end{bmatrix}$$

$$\begin{bmatrix} 0.1 & 0.2 & 0.3 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix} = \begin{bmatrix} 3.8 & 4.4 & 5 & 5.6 \end{bmatrix}$$

* *only 1 or*
 Multiply & ~~add~~ separate:
Infinite
 FMAC
 Pipelined MAC
 Maximum Parallel:

One Output

"higher"

24 cycles

24 cycles

24

All 4 Outputs

24

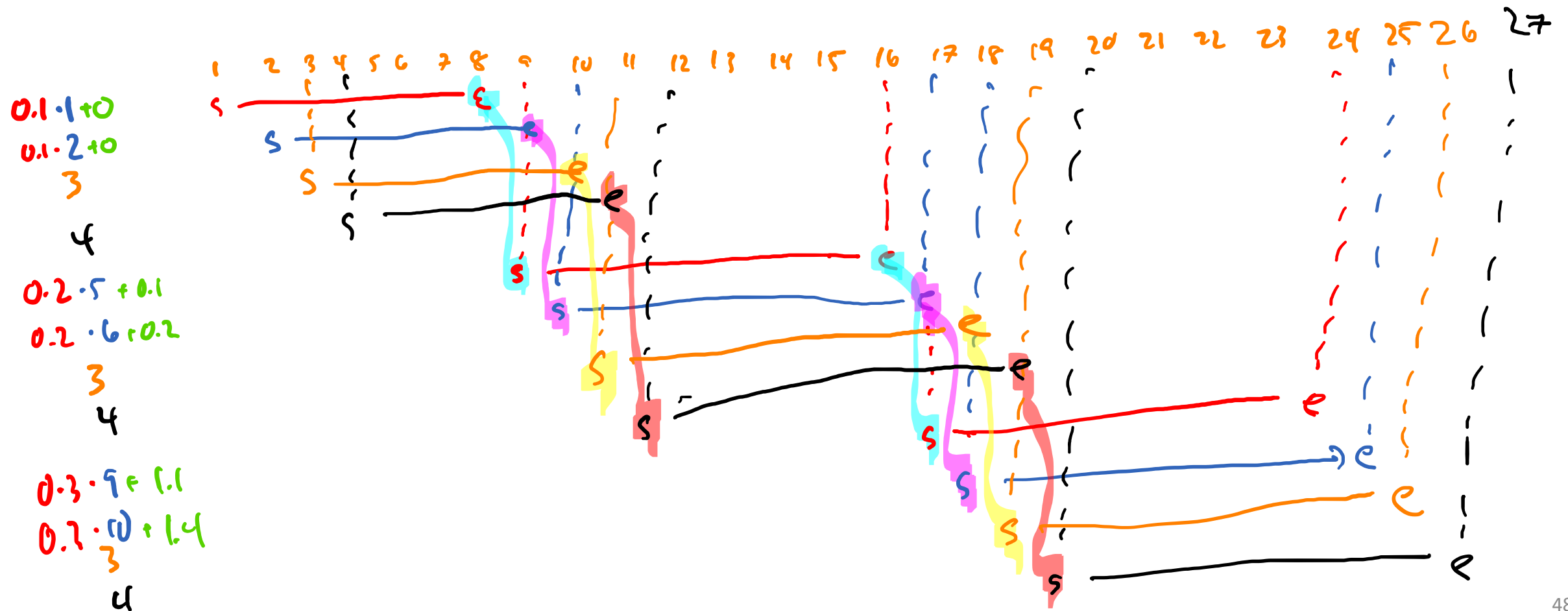
27 ~~cycles~~ cycles

24

$$\begin{bmatrix} 0.1 & 0.2 & 0.3 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix} = \begin{bmatrix} 3.8 & 4.4 & 5 & 5.6 \end{bmatrix}$$

	<u>One Output</u>	<u>All 4 Outputs</u>
Multiply & Add separate:	40 cycles	160 cycles
MAC	24 cycles	96 cycles
Pipelined MAC	24 cycles (+1 for extra rows)	33 cycles
Maximum Parallel:	24 cycles	24 cycles

Next Time: Divide and Conquer



$$\begin{bmatrix} 0.1 & 0.2 & 0.3 \end{bmatrix} \left[\begin{array}{cc|cc} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ \hline 9 & 10 & 11 & 12 \end{array} \right] = \begin{bmatrix} 3.8 & 4.4 & 5 & 5.6 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix}$$

$$0.1 \cdot \begin{bmatrix} 1 & 2 & 3 & 4 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.4 \end{bmatrix}$$

$$0.2 \cdot \begin{bmatrix} 5 & 6 & 7 & 8 \end{bmatrix} + \begin{bmatrix} 0.1 & 0.2 & 0.3 & 0.4 \end{bmatrix} = \begin{bmatrix} 1.1 & 1.4 & 1.7 & 2.0 \end{bmatrix}$$

$$\neq \begin{bmatrix} \underline{2.7} & \underline{3.0} & \underline{3.3} & \underline{3.6} \end{bmatrix}$$

$$0.3 \cdot \begin{bmatrix} 9 & 10 & 11 & 12 \end{bmatrix} + \begin{bmatrix} 1.1 & 1.4 & 1.7 & 2.0 \end{bmatrix} = \begin{bmatrix} 3.8 & 4.4 & 5 & 5.6 \end{bmatrix}$$

$$\begin{bmatrix} 3.8 & 4.4 & 5.0 & 5.6 \end{bmatrix}$$